

MTasking

Distribucion de Tareas y Bitacoras Segundo Parcial — Construccion de Software

Docente: PhD. Franklin Parrales

Fecha: Junio 2026

Instrucciones generales

1. Cada integrante debe implementar exactamente los archivos indicados en su seccion.
2. Cada integrante llena su propia bitacora con las actividades que realizo, fecha y horas.
3. El lider de proyecto (Daniel Mora) revisara el codigo antes de integrarlo al repositorio.
4. Los commits deben tener mensajes descriptivos: feat: descripcion, fix: descripcion.
5. No modificar archivos que no esten en tu lista sin coordinarlo primero con el lider.

Descripcion de la tarea

Implementar el flujo completo de recuperacion de contraseña olvidada. El usuario ingresa su email, el sistema genera un token unico con fecha de expiracion (1 hora), lo almacena en la tabla password_resets y envia un email con el enlace de restablecimiento usando mail() de PHP. El usuario hace clic en el enlace, ingresa una nueva contraseña y el sistema la actualiza. Validar que el token exista, no haya expirado y eliminar el registro tras su uso.

Criterios de aceptacion

- ✓ El email se envia correctamente con mail() de PHP nativo
- ✓ El token expira en 60 minutos
- ✓ El token se elimina de la BD despues de usarse
- ✓ Se valida que el nuevo password tenga minimo 6 caracteres
- ✓ Mensajes de error claros si el email no existe o el token expiro

Archivos a crear / modificar

Archivo	Accion	Descripcion breve
database/schema.sql	MODIFICAR	Agregar tabla password_resets (id, email, token, expires_at, created_at)
backend/controllers/AuthController.php	MODIFICAR	Agregar metodos forgotPassword() y resetPassword()
backend/routes/api.php	MODIFICAR	Agregar POST /auth/forgot-password y POST /auth/reset-password
frontend/html/index.html	MODIFICAR	Agregar enlace y formulario de recuperacion de contraseña
frontend/js/auth.js	MODIFICAR	Funciones forgotPassword() y resetPassword() con fetch al backend

Bitacora personal — a completar por el integrante

Completa esta tabla con cada sesion de trabajo. Sé específico en las actividades y archivos.

Fecha	Actividad realizada	Archivos creados / modificados	Horas	Observaciones

Firma del integrante:

Descripcion de la tarea

Implementar un sistema de comentarios de texto dentro del detalle de cada tarea. El usuario autenticado debe poder agregar un comentario, y el sistema debe mostrar todos los comentarios de la tarea ordenados por fecha (mas reciente primero), mostrando el nombre del autor y la fecha. Crear el modelo, controlador y rutas necesarias. Integrar la seccion de comentarios en el modal de detalle de tarea en el frontend.

Criterios de aceptacion

- ✓ Solo usuarios autenticados pueden comentar
- ✓ El comentario no puede estar vacio
- ✓ Se muestra nombre del autor y fecha/hora de cada comentario
- ✓ Los comentarios se cargan automaticamente al abrir el modal de la tarea
- ✓ El campo de texto se limpia despues de enviar exitosamente

Archivos a crear / modificar

Archivo	Accion	Descripcion breve
database/schema.sql	MODIFICAR	Agregar tabla task_comments (id, task_id, user_id, contenido, created_at)
backend/models/Comment.php	NUEVO	Metodos create(), getAllByTask() con JOIN a users
backend/controllers/CommentController.php	NUEVO	Metodos create() y listByTask() con validacion de sesion
backend/routes/api.php	MODIFICAR	GET /tasks/{id}/comments y POST /tasks/{id}/comments
frontend/html/project.html	MODIFICAR	Seccion de comentarios dentro del modal de detalle de tarea
frontend/js/tasks.js	MODIFICAR	Funciones loadComments() y addComment() con fetch

Bitacora personal — a completar por el integrante

Completa esta tabla con cada sesion de trabajo. Sé específico en las actividades y archivos.

Fecha	Actividad realizada	Archivos creados / modificados	Horas	Observaciones

Firma del integrante:

Descripcion de la tarea

Agregar el campo prioridad (Alta, Media, Baja) y fecha_limite (DATE) a la tabla tasks. Actualizar el modelo Task para incluir estos campos en create() y getAllByProject(). En el frontend mostrar un badge de color por prioridad (rojo=Alta, amarillo=Media, verde=Baja) y una alerta visual si la fecha limite ya paso. Ademas, agregar controles de filtrado en la vista del proyecto: filtrar por estado y/o prioridad tanto en el backend (query params) como en el frontend.

Criterios de aceptacion

- ✓ El campo prioridad tiene valor por defecto 'Media'
- ✓ Las tareas vencidas muestran indicador visual de alerta
- ✓ El filtro por estado funciona independientemente del filtro por prioridad
- ✓ Si no hay filtros activos se muestran todas las tareas
- ✓ Los nuevos campos aparecen en el formulario de creacion de tarea

Archivos a crear / modificar

Archivo	Accion	Descripcion breve
database/schema.sql	MODIFICAR	Agregar columnas prioridad ENUM y fecha_limite DATE a tabla tasks
backend/models/Task.php	MODIFICAR	Actualizar create(), getAllByProject() y findById() con nuevos campos
backend/controllers/TaskController.php	MODIFICAR	Leer prioridad y fecha_limite en create(); query params en listByProject()
backend/routes/api.php	MODIFICAR	Pasar query params a TaskController::listByProject()
frontend/html/project.html	MODIFICAR	Select prioridad + input fecha_limite en form; controles de filtro
frontend/js/tasks.js	MODIFICAR	Badges de color, alerta vencimiento, logica de filtrado

Bitacora personal — a completar por el integrante

Completa esta tabla con cada sesion de trabajo. Sé específico en las actividades y archivos.

Fecha	Actividad realizada	Archivos creados / modificados	Horas	Observaciones

Firma del integrante:

Descripcion de la tarea

Implementar un registro automatico cada vez que se cambia el estado de una tarea. Crear la tabla `task_history` con: quien hizo el cambio, de que estado vino y a cual fue, y cuando. El registro debe crearse automaticamente dentro de `TaskController::updateStatus()` sin que el usuario tenga que hacer nada extra. Exponer un endpoint `GET /tasks/{id}/history` y mostrar el historial en el modal de detalle de tarea como una linea de tiempo simple.

Criterios de aceptacion

- ✓ El historial se registra automaticamente en cada cambio de estado
- ✓ Se muestra el nombre del usuario que hizo el cambio
- ✓ Se muestra el estado anterior y el nuevo estado
- ✓ Los registros se muestran ordenados del mas reciente al mas antiguo
- ✓ Si no hay historial se muestra mensaje 'Sin cambios registrados'

Archivos a crear / modificar

Archivo	Accion	Descripcion breve
database/schema.sql	MODIFICAR	Agregar tabla <code>task_history</code> (<code>id</code> , <code>task_id</code> , <code>user_id</code> , <code>estado_anterior</code> , <code>estado_nuevo</code> , <code>created_at</code>)
backend/models/TaskHistory.php	NUEVO	Metodos <code>create()</code> y <code>getAllByTask()</code> con JOIN a <code>users</code>
backend/controllers/TaskController.php	MODIFICAR	Llamar <code>TaskHistory::create()</code> dentro de <code>updateStatus()</code> ; nuevo metodo <code>history()</code>
backend/routes/api.php	MODIFICAR	Agregar <code>GET /tasks/{id}/history</code>
frontend/html/project.html	MODIFICAR	Seccion historial en modal de detalle de tarea
frontend/js/tasks.js	MODIFICAR	Funcion <code>loadHistory()</code> y renderizado de linea de tiempo

Bitacora personal — a completar por el integrante

Completa esta tabla con cada sesion de trabajo. Sé específico en las actividades y archivos.

Fecha	Actividad realizada	Archivos creados / modificados	Horas	Observaciones

Firma del integrante:

Descripción de la tarea

Implementar dos funcionalidades de CRUD que faltan. Primero, edición completa de una tarea existente (título, descripción, responsable_id): agregar método update() al modelo Task, endpoint PUT /tasks/{id} en rutas y controlador, y un modal de edición en el frontend. Segundo, eliminación de proyectos: método delete() en el modelo Project, endpoint DELETE /projects/{id} con verificación de que el usuario es el dueño, y botón de eliminar en las tarjetas del dashboard con confirmación.

Criterios de aceptación

- ✓ Solo el creador del proyecto puede eliminarlo
- ✓ Al eliminar un proyecto se eliminan sus tareas en cascada (ya definido en schema)
- ✓ El modal de edición de tarea carga los datos actuales antes de editar
- ✓ La edición de tarea valida que el título no esté vacío
- ✓ El dashboard se actualiza automáticamente después de eliminar un proyecto

Archivos a crear / modificar

Archivo	Acción	Descripción breve
backend/models/Task.php	MODIFICAR	Agregar método update(id, título, descripción, responsable_id)
backend/controllers/TaskController.php	MODIFICAR	Agregar método update(id)
backend/models/Project.php	MODIFICAR	Agregar método delete(id)
backend/controllers/ProjectController.php	MODIFICAR	Agregar método delete(id) con verificación de ownership
backend/routes/api.php	MODIFICAR	PUT /tasks/{id} y DELETE /projects/{id}
frontend/html/project.html	MODIFICAR	Modal de edición de tarea
frontend/js/tasks.js	MODIFICAR	Funciones openEditTask(), saveTask() con fetch PUT
frontend/js/projects.js	MODIFICAR	Botón eliminar proyecto y función deleteProject()

Bitácora personal — a completar por el integrante

Completa esta tabla con cada sesión de trabajo. Sé específico en las actividades y archivos.

Fecha	Actividad realizada	Archivos creados / modificados	Horas	Observaciones

Firma del integrante:

Descripción de la tarea

Dos tareas de calidad de código. Primera: refactorizar `ProjectController` y `TaskController` para que usen la jerarquía `AppException` que ya existe en el proyecto (`AuthController` ya la usa como referencia). Reemplazar todos los `http_response_code()` + `echo json_encode(['error'...])` directos por bloques `try/catch` con `AppException` o `RecursoNoEncontradoException` según corresponda. Segunda: escribir tests de integración con `PHPUnit` para los tres controladores, usando la misma base `SQLite` en memoria de `DatabaseTestCase.php` ya existente.

Criterios de aceptación

- ✓ `ProjectController` y `TaskController` no tienen llamadas directas a `http_response_code()`
- ✓ Todos los errores pasan por `AppException` y su método `toJson()`
- ✓ Tests cubren: crear, listar, detalle, actualizar y eliminar en proyectos y tareas
- ✓ Tests cubren flujos de error: recurso no encontrado, no autenticado
- ✓ Todos los tests pasan con `php phpunit.phar`

Archivos a crear / modificar

Archivo	Acción	Descripción breve
backend/controllers/ProjectController.php	MODIFICAR	Reemplazar manejo de errores manual por <code>AppException</code>
backend/controllers/TaskController.php	MODIFICAR	Reemplazar manejo de errores manual por <code>AppException</code>
tests/Unit/ProjectControllerTest.php	NUEVO	Tests de integración para endpoints de proyectos
tests/Unit/TaskControllerTest.php	NUEVO	Tests de integración para endpoints de tareas
tests/Unit/AuthControllerTest.php	NUEVO	Tests de integración para endpoints de autenticación

Bitácora personal — a completar por el integrante

Completa esta tabla con cada sesión de trabajo. Sé específico en las actividades y archivos.

Fecha	Actividad realizada	Archivos creados / modificados	Horas	Observaciones

Firma del integrante:

Descripcion de la tarea

Dos tareas. Primera (codigo): implementar la edicion de perfil de usuario. El usuario autenticado debe poder cambiar su nombre y su contraseña desde una pagina de perfil. Para cambiar la contraseña se debe verificar la contraseña actual primero. Crear el endpoint PUT /auth/profile, actualizar el modelo User y agregar la pagina de perfil al frontend. Segunda (documento): redactar el documento de Politica de Mantenimiento del sistema MTasking cubriendo mantenimiento correctivo, preventivo y evolutivo, responsables y periodicidad.

Criterios de aceptacion

- ✓ Se requiere contraseña actual para cambiar la contraseña
- ✓ El nombre no puede quedar vacio
- ✓ Si solo se cambia el nombre, la contraseña no se toca
- ✓ La sesion refleja el nombre actualizado inmediatamente
- ✓ El documento de politica cubre los 3 tipos de mantenimiento con responsables

Archivos a crear / modificar

Archivo	Accion	Descripcion breve
backend/models/User.php	MODIFICAR	Agregar metodo update(id, nombre, password_hash)
backend/controllers/AuthController.php	MODIFICAR	Agregar metodo updateProfile()
backend/routes/api.php	MODIFICAR	Agregar PUT /auth/profile
frontend/html/profile.html	NUEVO	Pagina de perfil con formulario de edicion
frontend/js/profile.js	NUEVO	Logica de carga y actualizacion de perfil
docs/politica_de_mantenimiento.pdf	NUEVO	Documento PDF de politica de mantenimiento

Bitacora personal — a completar por el integrante

Completa esta tabla con cada sesion de trabajo. Sé específico en las actividades y archivos.

Fecha	Actividad realizada	Archivos creados / modificados	Horas	Observaciones

Firma del integrante:

Descripcion de la tarea

Dos tareas. Primera (codigo): implementar un endpoint de estadísticas por proyecto. GET /projects/{id}/stats debe devolver: total de tareas, cuantas estan Pendiente, En progreso y Terminado, y opcionalmente el porcentaje de completitud. En el frontend mostrar estas cifras en la vista del proyecto como tarjetas de resumen o una barra de progreso antes de la lista de tareas. Segunda (documento): redactar el Manual de Usuario completo con capturas de pantalla o mockups de cada flujo: registro, login, proyectos y tareas.

Criterios de aceptacion

- ✓ El endpoint devuelve conteos correctos incluso si hay 0 tareas
- ✓ Las metricas se actualizan automaticamente al cambiar el estado de una tarea
- ✓ El frontend muestra los 3 estados con su respectivo conteo
- ✓ El manual cubre todos los flujos principales del sistema
- ✓ El manual incluye capturas o mockups de cada pantalla

Archivos a crear / modificar

Archivo	Accion	Descripcion breve
backend/controllers/ProjectController.php	MODIFICAR	Agregar metodo stats(id)
backend/routes/api.php	MODIFICAR	Agregar GET /projects/{id}/stats
frontend/html/project.html	MODIFICAR	Seccion de tarjetas de metricas sobre la lista de tareas
frontend/js/tasks.js	MODIFICAR	Funcion loadStats() y renderizado de metricas
docs/manual_de_usuario.pdf	NUEVO	Manual de usuario con flujos y capturas

Bitacora personal — a completar por el integrante

Completa esta tabla con cada sesion de trabajo. Sé específico en las actividades y archivos.

Fecha	Actividad realizada	Archivos creados / modificados	Horas	Observaciones

Firma del integrante:

Responsabilidades

- ✓ Revisar y aprobar el codigo de cada integrante antes de mergear al repositorio
- ✓ Verificar que los endpoints sean consistentes con la arquitectura existente
- ✓ Corregir errores de integracion entre modulos
- ✓ Redactar el Manual Tecnico del sistema
- ✓ Coordinar la entrega final y asegurar que el sistema corre localmente

Bitacora personal — Lider de Proyecto

Fecha	Actividad realizada	Archivos creados / modificados	Horas	Observaciones

Firma del integrante:
